Sonar debris detection on the edge: data split, model and runtime

sonar_edge project notes

3 November 2026

Abstract

This note documents the three parts of `sonar_edge` that are easiest to get wrong when reading the results: how the 1 868 forward-looking sonar frames are split into scene folds for cross-validation (Section 2), what the detector computes and how it is trained (Section 3), and how the quantised network runs in a standard-library-only C runtime that is bit-exact with the Python reference (Section 4). Section 5 collects the current numbers. The per-milestone documents in `docs/M0`–`M4` contain the full method and discussion; this is the overview.

1 Data

The dataset is the water-tank subset of the marine debris FLS collection of Valdenegro-Toro et al.: 1 868 frames from an ARIS Explorer 3000, projected to Cartesian coordinates at 320×480 pixels (width $\times$ height), single channel, 8-bit, with Pascal-VOC boxes in 11 classes (3 882 boxes, 1 to 5 per frame). Ten classes are debris objects; the eleventh, *Wall*, marks the tank walls and is kept as a detector class (a deployed system must not report walls as debris) but is excluded from scene matching because walls are static and would link unrelated recordings.

The frames come from a handful of recordings in which the sonar drifts slowly over a fixed arrangement of objects, so consecutive frames are near-duplicates. The published frame indices are shuffled: the median thumbnail correlation of index-adjacent pairs (0.757) equals that of random pairs (0.747), and only 2% of adjacent pairs have fully matching box layouts, again the same as random pairs. Any split by frame therefore places near-identical frames on both sides and the test score measures memorisation. This is the leak the predecessor study (`sonar_cnn`) documented for cropped patches; whole-frame detection inherits it.

2 Scene reconstruction and folds

2.1 Linking rule

Since recording order is lost, scenes are reconstructed from the annotations. For two frames A and B, first drop the Wall boxes. Define the *layout match*

$$m(A \to B) \;=\; \frac{1}{|A|}\,\big|\{\, p \in A \;:\; \exists\, q \in B,\ \mathrm{cls}(q) = \mathrm{cls}(p),\ \mathrm{IoU}(p,q) \geq 0.5 \,\}\big|, \tag{1}$$

the fraction of A's boxes that have a same-class box in B at the same place. IoU is intersection over union in original pixel coordinates. A and B are *linked* iff

$$m(A \to B) = 1 \quad \text{and} \quad m(B \to A) = 1, \tag{2}$$

i.e. every object in each frame has a counterpart in the other. Frames with no non-wall boxes link to nothing. Scenes are the connected components of the link graph (union–find over all $\binom{1868}{2}$

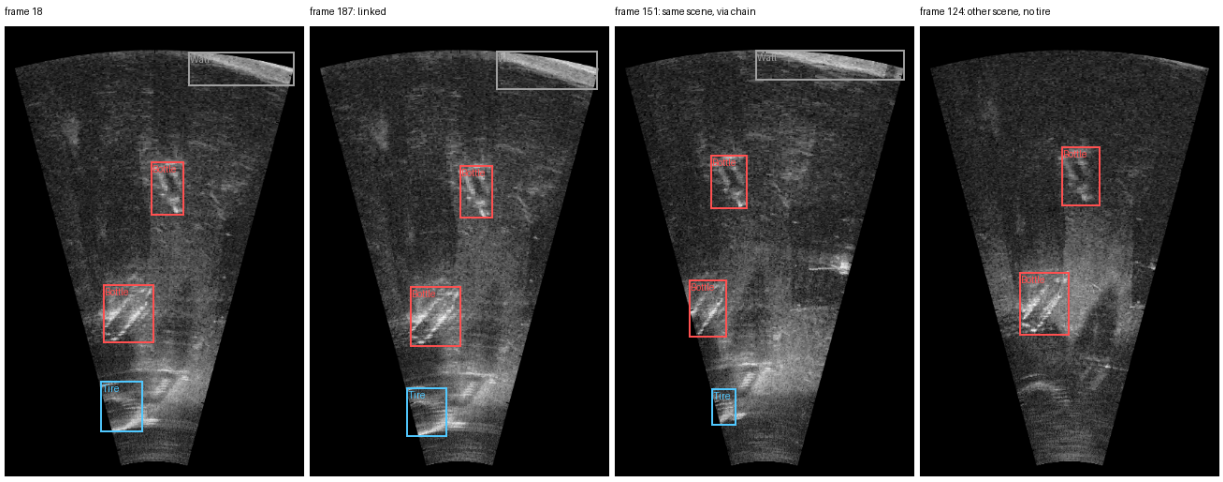


Figure 1: Frame 18 and three partners. Frames 18 and 187 are linked directly (every box matches at IoU ≥ 0.5). Frame 151 belongs to the same scene but is not linked to frame 18 directly: the sonar has drifted and the boxes no longer overlap enough; the link runs through intermediate frames. Frame 124 shows the same two bottles but no tire, so the match from 18 fails ($2/3 < 1$); it is a different scene and lies in a different fold.

Pair	Box in frame 18 (x, y, w, h)	best same-class IoU	$m(18 \to B)$	$m(B \to 18)$	verdict
18, 187	Bottle (156,144,35,57) Bottle (105,275,54,62) Tire (102,378,45,54)	0.71 0.85 0.76	3/3	3/3	linked
18, 151	Bottle (156,144,35,57) Bottle (105,275,54,62) Tire (102,378,45,54)	0.00 0.16 0.42	0/3	0/3	not linked directly; same scene via chain (scene 176, fold 4)
18, 124	Bottle (156,144,35,57) Bottle (105,275,54,62) Tire (102,378,45,54)	0.56 0.71 –	2/3	2/2	not linked; scene 10, fold 3

Table 1: Layout matching for the pairs of Figure 1. A pair is linked only when both fractions are 1.

pairs). Linkage is transitive on purpose: a long recording drifts, and its first and last frames need not match directly.

The two strict choices are deliberate. Requiring *every* box in *both* directions stops chaining through one shared object (two different scenes with a tire in the same corner must not merge). Excluding walls matters more than it looks: with walls included, or with the required fraction relaxed from 1 to 0.5, 1 686 of the 1 868 frames collapse into a single component.

The procedure yields **191 scenes**. The largest have 201, 158, 158, 119 and 90 frames; 118 scenes are single frames, mostly frames whose only boxes are walls. Random members of four scenes were inspected and each is one physical arrangement seen through small viewpoint and gain changes.

2.2 Worked example

Figure 1 shows four frames; Table 1 gives the matching arithmetic from frame 18 to each of the others.

Frame 124 also shows the method's known weakness. It is plausibly the same physical setup photographed before the tire was placed; the rule cannot know that, cuts it into a separate scene, and that scene happens to land in another fold. Shared background therefore leaks a little between

folds. The error is in the conservative direction (too few links make the evaluation stricter, never looser), which is why it is accepted.

2.3 Folds

The 191 scenes are dealt into $K = 5$ folds by a deterministic greedy procedure: scenes are taken largest first, and each goes to the fold that minimises, after the assignment, the sum over classes of the squared share of that class's boxes plus the squared share of frames. A fold that already holds much of a class, or many frames, is penalised. The result is committed as `splits/scene_folds_v1.json` (frame → scene, fold).

	fold 0	fold 1	fold 2	fold 3	fold 4
scenes	34	39	38	43	37
frames	439	388	383	230	428
Bottle	43%	12%	6%	3%	36%
Can	22%	18%	24%	3%	33%
Chain	14%	15%	22%	37%	11%
Drink-carton	24%	54%	6%	2%	14%
Hook	23%	30%	13%	6%	28%
Propeller	32%	23%	22%	5%	18%
Shampoo-bottle	7%	10%	35%	5%	42%
Standing-bottle	3%	0%	3%	94%	0%
Tire	34%	39%	9%	3%	15%
Valve	2%	17%	66%	5%	10%
Wall	17%	25%	16%	18%	24%

Table 2: Fold sizes and the share of each class's boxes per fold.

Balance is limited by the data: several classes live almost entirely in one scene (94% of standing-bottle boxes, 66% of valve, 54% of drink-carton), and a scene cannot be split. Fold 3 is small because it carries the standing-bottle scene.

2.4 Protocols

Development. Fold 0 is the test set, fold 1 the validation set (used only to pick the best epoch), folds 2–4 the training set (1 041 / 388 / 439 frames). All iteration happens here.

Final: 5-fold cross-validation. For $k = 0, \ldots, 4$ a model is trained from scratch with fold k as test, fold $(k + 1)$ mod 5 as validation and the remaining three as training. Run 2, for instance, trains on folds 4, 0, 1, selects on fold 3 and tests on fold 2. Every scene is tested exactly once by a model that never saw a frame of it. The five test prediction sets partition the dataset, so they are merged and scored as *one* detection problem (*pooled AP*, VOC convention). Pooling is the only estimate that is defined for the one-scene classes: standing-bottle is testable only in the run that holds out fold 3, where the training set contains four standing bottles, and pooling reports that honestly instead of averaging in four undefined values. The mean and standard deviation of the per-fold mAP are reported alongside as a measure of fold-to-fold variation, which is large because the folds contain different scenes. Both the float model and its int8 quantisation are pooled, since the C runtime reproduces the latter exactly (Section 4).

Noise. Training is not bit-reproducible on the GPU. A seed-identical repeat of the shipped configuration on the development split moved the test mAP by 3.8 points, so single-split differences below about 4 points mean nothing; only pooled cross-validation separates configurations.

3 Model

3.1 Architecture

The detector is a single-stage, anchor-free network in the CenterNet style (Zhou et al., 2019), reduced to what a tiny C runtime can execute: a plain stack of 3×3 convolutions with batch normalisation and ReLU, no residual connections, no upsampling, and a 1×1 head. Output stride is 8, so a 480×320 input yields a 60×40 grid and a 240×160 input a 30×20 grid. Dilation in the later layers grows the receptive field to about 255 input pixels without further downsampling.

Config	Layers (channels; s2 = stride 2, dk = dilation k)	Input	Grid	Params	MACs	RF
`small`	16s2, 32s2, 32s2, 32, 32d2, 32d4	240×160	30×20	42,463	34.8 M	254
`small @ 4/3`	same	360×240	45×30	42,463	78.4 M	254
`small_wide`	16s2, 32s2, 64s2, 64, 64d2, 64d4	240×160	30×20	135,359	90.4 M	254
`base`	16s2, 32s2, 48s2, 48, 48d2, 48d4, 48d8	480×320	60×40	102,831	283.7 M	255
`wide`	16s2, 32s2, 64s2, 64, 64d2, 64d4, 64d8	480×320	60×40	172,351	450.2 M	255

Table 3: Configurations. Every layer is conv 3×3 + BN + ReLU (stride 1, dilation 1 unless marked); the head is a 1×1 convolution to $C + 4 = 15$ channels. RF is the receptive field in input pixels of the full-resolution frame. MACs are per frame. `small` is the shipped model.

Per grid cell the head predicts 15 values: $C = 11$ class logits (a heatmap), the box size (w, h) in units of the stride, and the centre offset $(\delta_x, \delta_y) \in [0, 1)$ within the cell. The heatmap bias is initialised to -2.19 (initial probability ≈ 0.01), as in CenterNet, to keep the focal loss stable at the start. Inputs are the raw 8-bit pixels scaled to $[0, 1]$; the half-resolution configurations average 2×2 blocks, and the 4/3 scale uses a box-filter resize.

3.2 Targets and losses

For each ground-truth box with centre (c_x, c_y) and size (w, h) in grid units, the target heatmap of its class receives a Gaussian splat (max-merged) centred on the integer cell $(\lfloor c_x \rfloor, \lfloor c_y \rfloor)$ with the CornerNet radius, the smallest r such that a box shifted by r still has IoU ≥ 0.7 with the original. The size and offset targets are stored at that cell only. The loss is

$$\mathcal{L} = \mathcal{L}_{\text{heat}} + 0.1\, \mathcal{L}_{\text{size}} + \mathcal{L}_{\text{off}}, \tag{3}$$

where $\mathcal{L}_{\text{heat}}$ is the penalty-reduced pixelwise focal loss of CenterNet ($\alpha = 2$, $\beta = 4$, normalised by the number of objects) over the sigmoid of the logits, and the two regression terms are L1 at the object cells, normalised by the number of objects.

3.3 Decoding

Detections are the local maxima of the logit heatmap: a cell is kept if it is $\geq$ all of its 3×3 neighbours (a max-pool comparison, no NMS). The top $K = 32$ peaks over all classes are taken, those below the logit threshold -4 (probability ≈ 0.018) dropped, and each is turned into a box

$$c_x = (i_x + \delta_x)\, s, \qquad c_y = (i_y + \delta_y)\, s, \qquad \text{box} = \left[c_x - \tfrac{ws}{2},\ c_y - \tfrac{hs}{2},\ c_x + \tfrac{ws}{2},\ c_y + \tfrac{hs}{2}\right], \tag{4}$$

with s the stride. The decoder works on logits because the sigmoid is monotone, which lets the C decoder threshold the integer head output directly.

3.4 Training

AdamW (weight decay 10^{-4}), one-cycle learning-rate schedule with peak 2×10^{-3} and 10% warm-up, batch size 16, seed 0, 150 epochs for the final models. The best epoch is selected on validation

mAP and the test fold is evaluated once with it. Augmentation (*strong*): horizontal flip, integer translation up to 8 px, random zoom by a factor in $[0.75, 1.3]$ about a random point with probability 0.7 (boxes scaled, clipped and dropped when less than 30% or 4 px remain), additive Gaussian noise with $\sigma \sim U[0, 0.04]$, multiplicative gain in $[0.7, 1.3]$ and additive bias in $[-0.1, 0.1]$. Strong augmentation was worth about +5 mAP over the basic set on the development split; a longer schedule alone was worth nothing.

The metric is VOC AP at IoU 0.5 with all-point interpolation, computed per class and averaged over the classes present (own implementation in `metrics.py`, reused unchanged to score the C binary).

4 Quantisation and runtime

4.1 Post-training int8

Batch normalisation is folded into the preceding convolution ($W' = W\gamma/\sqrt{\sigma^2+\epsilon}$, $b' = \beta - \mu\gamma/\sqrt{\sigma^2+\epsilon}$ per channel), so the runtime has no normalisation layer. Every activation is either a raw pixel or a ReLU output, hence non-negative, and is stored as `uint8` with zero point 0 and one scale s_a per layer; weights are symmetric `int8` with one scale per output channel, $s_{w,c} = \max|W_c|/127$. A layer computes, per output channel c,

$$\text{acc} = \sum_i W_{q,c,i}\, a_{q,i} + b_{q,c}, \qquad b_{q,c} = \text{round}\Big(\frac{b'_c}{s_{w,c}\, s_{\text{in}}}\Big), \qquad \text{out}_q = \text{clip}\Big(\text{round}\big(\text{acc}\cdot\tfrac{s_{w,c}\, s_{\text{in}}}{s_{\text{out}}}\big),\, 0,\, 255\Big), \tag{5}$$

where the clip is the ReLU. The real multiplier is replaced by an `int32` $M_c \in [2^{30}, 2^{31})$ and a shift n_c (relative error $< 10^{-8}$), so requantisation is one `int64` multiply, an add of 2^{n_c-1} (round half away from zero) and a shift. The 1×1 head is not requantised: its `int32` accumulators are returned with their per-channel real scale and decoded directly. Activation scales come from the 99.99th percentile of post-ReLU outputs over 64 training frames, divided by 255.

A Python reference evaluates these integer convolutions as `float64` tensor operations, which is exact (every product $< 2^{15}$, every sum has $< 2^{12}$ terms, all intermediates are integers far below 2^{53}), and so provides a bit-exact specification that is fast enough to score the whole test fold. It also exports the weights and three golden input/output pairs.

4.2 C runtime

The runtime is C11, standard library only, built with GNU make (`-std=c11 -O2 -Wall -Wextra -Werror -pedantic`), about 165 lines plus the generated `weights.h` (int8 kernels, int32 biases, (M_c, n_c) pairs, layer table and geometry as preprocessor constants). All memory is static: two activation buffers that alternate as input and output, and the int32 head map. Weights occupy 42 KB for `small`.

Activations are stored HWC (channels innermost) with a zero border equal to the *consuming* layer's dilation d, so a 3×3 convolution with stride s reads, for output (o_y, o_x) and tap (k_y, k_x), the padded pixel $(o_y s + k_y d,\ o_x s + k_x d)$ with no bounds check, reproducing PyTorch's `padding = dilation`. Each tap is a dot product over a contiguous run of c_{in} bytes against a contiguous run of c_{in} weights, the access pattern a NEON inner loop wants. Accumulation is `int32` ($|\text{acc}| < 2^{28}$, an 8× margin), requantisation in `int64`, and only non-negative quantities are ever shifted, so the implementation-defined right shift of a negative integer in C is never used.

The decoder scans each class's int32 logit map for cells that are $\geq$ their existing 3×3 neighbours (matching `max_pool2d` with its implicit $-\infty$ padding), converts the score to a real value in the same operation order as the Python reference, and forms the box in double precision with the same expression order as the Python decoder. These per-detection scalars are the only floating-point operations in the runtime. The interface is

```
int se_detect(const uint8_t *frame, float thr, se_detection_t *out, int max_out);
const int32_t *se_forward(const uint8_t *frame);   /* int32 head map, HWC */
```

and the CLI `sonar-detect` reads 8-bit PGM frames and prints one detection per line.

4.3 Verification

Two checks close the chain. `make test` runs the golden test: the C head map on the three exported frames equals the Python int8 reference on every one of the $30 \times 20 \times 15$ values. `sonar-edge c_eval` then runs the binary over the whole test fold and scores it with the same metric and ground truth as the Python models: 65.17 mAP in C against 65.16 for Python int8 on the development test fold (the residual is tie ordering among equal scores). The accuracy of the deployed system is therefore established by the Python cross-validation and does not need re-measuring on the device; only speed and energy do.

On one core of a laptop CPU (Ryzen 5 5600H, scalar C, `-O2`) the shipped `small` model takes 10 ms per frame, about 3.5 GMAC/s. The Raspberry Pi estimate of 35–50 ms per frame (M5) awaits hardware.

5 Results

5.1 Development split

Fold 0 test, fold 1 validation, strong augmentation, 150 epochs. These are the numbers used to choose the configuration; they are single runs and carry the 4-point noise of Section 2.

Config	Input	MACs	Val mAP	Test mAP (float)	Test mAP (int8 / C)
`base`	480×320	284 M	66.1	63.1	–
`small` (shipped)	240×160	35 M	55.0	64.1	65.2 / 65.17
`wide`	480×320	450 M	67.8	68.8	–
`small`, seed-identical repeat	240×160	35 M	59.9	60.3	–

Table 4: Development-split results (mAP@0.5, %).

5.2 5-fold cross-validation

Pooled AP over all 1 868 frames, every scene tested once. The per-fold values are listed to show which differences are real: `wide` beats `small` on every fold by 3–9 points, `small_wide` is inside the noise.

Config	MACs	Pooled float	Pooled int8	Fold mean (s.d.)	Per-fold float, folds 0–4
`small` (shipped)	35 M	49.4	49.7	56.4 (8.9)	60.3, 59.3, 49.9, 44.9, 67.4
`small` @ 360×240	78 M	running (folds 0–2 so far: 64.0, 66.5, 52.3)			
`small_wide`	90 M	50.2	49.6	58.0 (11.9)	60.1, 63.3, 51.1, 42.2, 73.4
`wide`	450 M	55.2	56.2	62.8 (8.3)	66.9, 68.6, 55.7, 52.1, 70.5

Table 5: Cross-validation results (mAP@0.5, %).

Reading the numbers. The pooled values are much lower than the development-split ones because fold 0 is an easy fold and folds 2 and 3 hard; the fold ranking is the same for every configuration, so the spread measures the data, not the model. `wide`'s gain is in the small and low-contrast classes (drink-carton, can, standing-bottle, bottle, tire), which are the objects that shrink to near the 8-px stride at half resolution. Doubling the channel width at half resolution

Class (boxes)	`small` float	`small` int8	`small_wide` float	`wide` float	`wide` int8
Bottle (468)	78.0	77.4	55.9	84.7	82.7
Can (292)	9.1	9.6	8.3	18.8	20.5
Chain (320)	32.5	32.3	38.3	29.8	30.2
Drink-carton (347)	62.4	64.1	69.2	81.4	79.8
Hook (171)	90.8	91.4	95.0	95.5	94.9
Propeller (197)	51.2	50.9	47.6	46.7	46.9
Shampoo-bottle (111)	27.7	25.5	42.0	30.5	31.4
Standing-bottle (65)	1.2	2.5	2.0	24.1	26.7
Tire (615)	86.8	86.8	84.0	93.9	92.7
Valve (240)	22.8	23.2	28.8	17.5	27.5
Wall (1056)	81.3	83.4	81.0	84.8	84.9
mAP pooled	49.4	49.7	50.2	55.2	56.2

Table 6: Pooled per-class AP (%).

(`small_wide`) recovers none of it, so the gain is a resolution effect; the running 360×240 experiment tests how much of it is available within the ∼100 M MAC budget of the Pi. Quantisation is neutral for `small` and `wide` (±1 pooled) and costs `small_wide` about 1–4 points per fold, for reasons not yet isolated.

6 Status

Milestones M0–M4 (data and protocol, detector, quantisation, C runtime, end-to-end pipeline) are complete and documented in `docs/`. The shipped artefact is `runtime/weights.h` for `small`, strong augmentation, 150 epochs, trained on the development split. M5, timing and energy on the Raspberry Pi, is blocked on hardware; the runtime builds on the laptop and cross-compiles with `make CC=aarch64-linux-gnu-gcc`.